Walchand College of Engineering, Sangli

Robotics Laboratory — Assignment 8

Implementation of Dobot Magician with Waypoints and Sensor Guide

Name - Atharv Kulkarni

PRN - 245109074

Class - TY CSE(B)

GitHub Link:
https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/Assignment_8/Session5_Waypoints(1).ipynb

https://github.com/Atharv-K-979/WCE-Labs-Material/blob/main/TY_Robotics_Lab/Assignment_8/Session6_Sensor_Guided(1).ipynb

Experiment Title:

Implementation of Dobot Magician with Waypoints and Sensor Guide

AIM:

To implement waypoint-based robotic motion and sensor-guided manipulation by planning a sequence of object pick-and-place actions and using detected object positions to guide the robot.

OBJECTIVES:

- To initialize the simulation environment and load the robotic arm.
- To program a sequence of pick-and-place actions using waypoints for multiple objects.
- To compare the original object order with a reordered sequence and analyze path distance and execution time.
- To simulate object-position detection and use the detected position instead of a manually hard-coded position.
- To test sensor-guided manipulation across multiple randomized object positions and consider the effects of inaccurate sensor readings.

THEORY:

Waypoint programming specifies a sequence of positions through which a robot moves while carrying out a task. A pick-and-place trajectory typically includes approach, grasp, lift, transfer, lower, and release stages. For multiple objects, the same routine can be reused in a loop. Reordering the objects may reduce total travel distance and cycle time, depending on the chosen sequence and the robot's motion.

Sensor-guided manipulation uses a perception step to estimate an object's location before planning the robot's action. Rather than relying on a fixed position entered manually, the manipulation routine receives the detected position and moves toward it. In a real robot, cameras or other sensors may provide this information; sensor calibration, noise filtering, and coordinate transformations are important for reliable grasping.

EXPERIMENT STEPS WITH SCREENSHOT OF EACH STEP:

PART A: IMPLEMENTATION OF DOBOT MAGICIAN WITH WAYPOINTS

Step 1: Installation and Environment Setup

Install PyBullet and import the required libraries. Initialize the simulation environment, load the ground plane and robotic arm, and prepare the helper functions for snapshots and pick-and-place actions.



Step 2: Code Implementation

Define the list of objects with their start and target positions. Execute the pick-and-place routine for each object using a loop. Reorder the object list and calculate/compare the total path distance and elapsed time for the original and optimized sequences.

```python
client = None
robot_id = None
plane_id = None

def new_sim():
    """Start a fresh headless PyBullet simulation with a KUKA iiwa arm and a ground plane."""
    global client, robot_id, plane_id
    if client is not None:
        p.disconnect(client)
    client = p.connect(p.DIRECT)  # headless — no local GUI needed
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane_id = p.loadURDF("plane.urdf")
    robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
    for _ in range(20):
        p.stepSimulation()
    return robot_id

def snapshot(cam_target=(0, 0, 0.4), cam_dist=1.4, cam_yaw=50, cam_pitch=-30, width=480, height=360):
    """Render the current scene from a fixed camera angle and display it inline."""
    view_matrix = p.computeViewMatrixFromYawPitchRoll(cam_target, cam_dist, cam_yaw, cam_pitch, 0, 2)
    proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=width/height, nearVal=0.1, farVal=3.0)
    img = p.getCameraImage(width, height, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
    rgb = np.reshape(img[2], (height, width, 4))[:, :, :3]
    plt.figure(figsize=(6, 4.5))
    plt.imshow(rgb)
    plt.axis('off')
    plt.show()

def get_num_joints():
    return p.getNumJoints(robot_id)

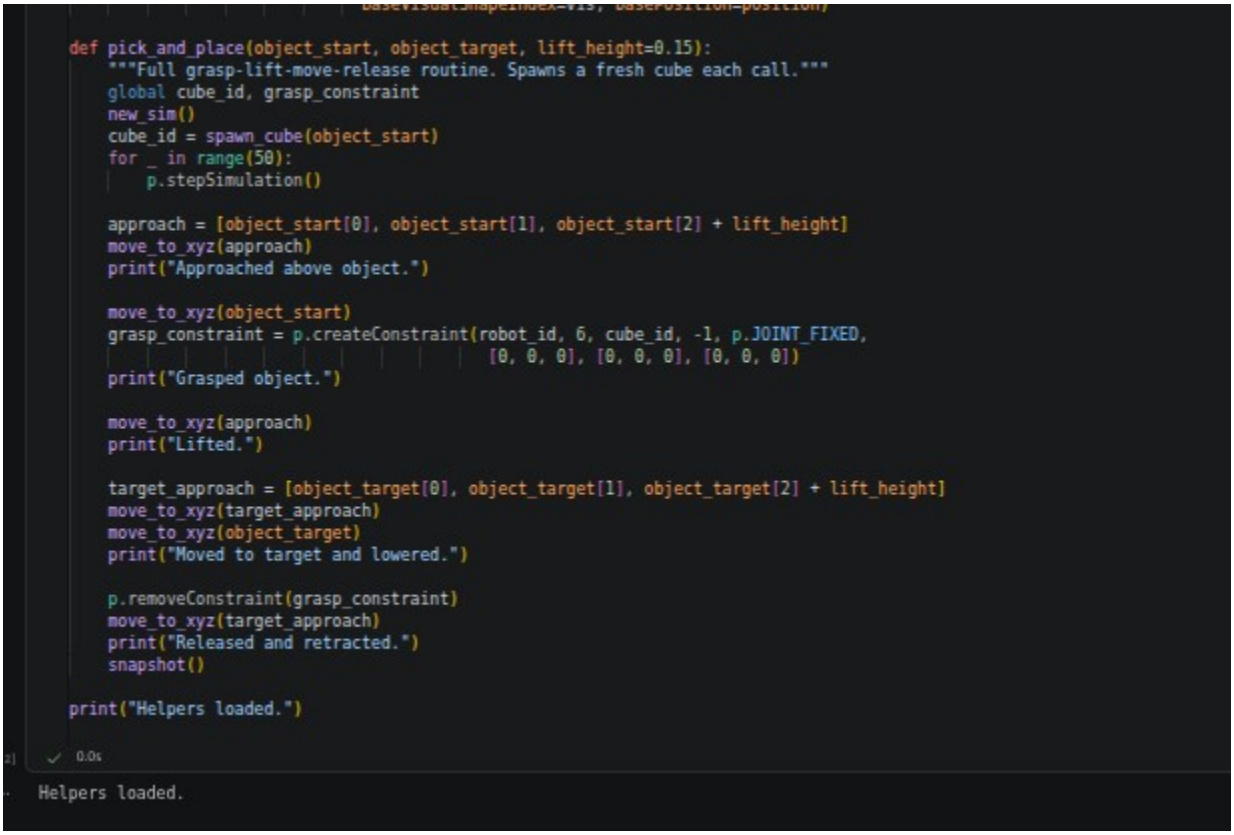
def get_joint_info():
    info = []
    for i in range(p.getNumJoints(robot_id)):
        ji = p.getJointInfo(robot_id, i)
        info.append({"index": ji[0], "name": ji[1].decode(), "type": ji[2]})
    return info

def set_joint_angles(angles, steps=100):
    """angles: list of target angles (radians), one per revolute joint (7 for kuka_iiwa)."""
    for i, a in enumerate(angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()

def get_end_effector_position():
    """KUKA iiwa end-effector is link index 6."""
    state = p.getLinkState(robot_id, 6)
    return state[0]  # (x, y, z)

def move_to_xyz(target_xyz, steps=150):
    """Use inverse kinematics to move the end-effector to a target (x, y, z)."""
    joint_angles = p.calculateInverseKinematics(robot_id, 6, target_xyz)
    for i, a in enumerate(joint_angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()
    return get_end_effector_position()

def spawn_cube(position, half_extent=0.03, color=(1, 0, 0, 1)):
    col = p.createCollisionShape(p.GEOM_BOX, halfExtents=[half_extent]*3)
    vis = p.createVisualShape(p.GEOM_BOX, halfExtents=[half_extent]*3, rgbaColor=color)
    return p.createMultiBody(baseMass=0.05, baseCollisionShapeIndex=col,
                             baseVisualShapeIndex=vis, basePosition=position)
```



```
def pick_and_place(object_start, object_target, lift_height=0.15):
    """Full grasp-lift-move-release routine. Spawns a fresh cube each call."""
    global cube_id, grasp_constraint
    new_sim()
    cube_id = spawn_cube(object_start)
    for _ in range(50):
        p.stepSimulation()

    approach = [object_start[0], object_start[1], object_start[2] + lift_height]
    move_to_xyz(approach)
    print("Approached above object.")

    move_to_xyz(object_start)
    grasp_constraint = p.createConstraint(robot_id, 6, cube_id, -1, p.JOINT_FIXED,
                                          [0, 0, 0], [0, 0, 0], [0, 0, 0])
    print("Grasped object.")

    move_to_xyz(approach)
    print("Lifted.")

    target_approach = [object_target[0], object_target[1], object_target[2] + lift_height]
    move_to_xyz(target_approach)
    move_to_xyz(object_target)
    print("Moved to target and lowered.")

    p.removeConstraint(grasp_constraint)
    move_to_xyz(target_approach)
    print("Released and retracted.")
    snapshot()

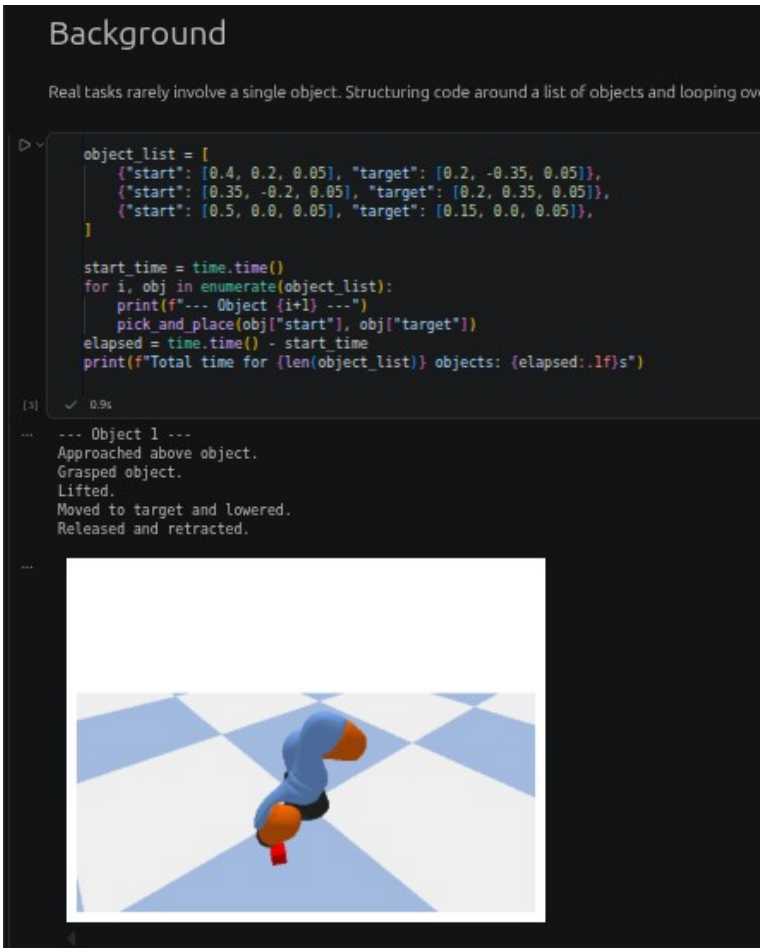
print("Helpers loaded.")
```

0.0s

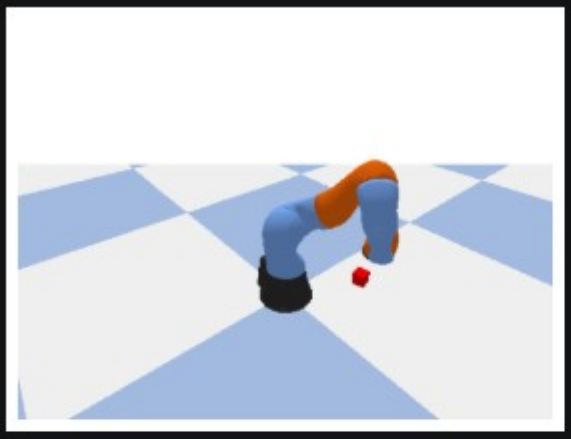
```
Helpers loaded.
```

Step 3: Output

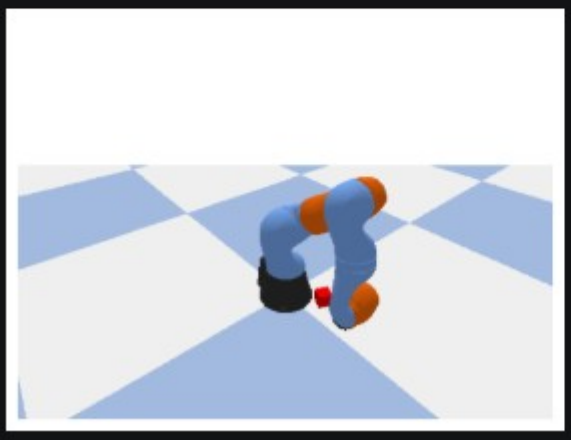
Run the original and reordered waypoint sequences. Capture the simulation snapshots and the printed path-distance and execution-time comparison.



```
--- Object 2 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
--- Object 3 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```

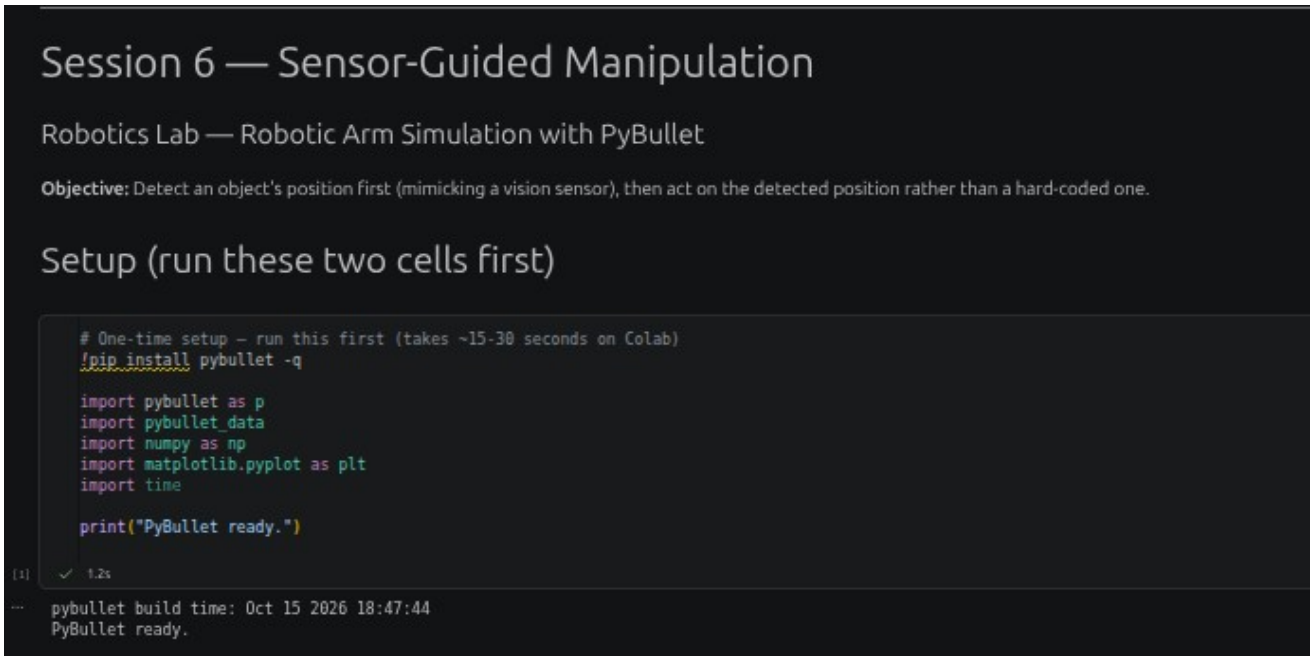


```
Total time for 3 objects: 1.0s
```

PART B: IMPLEMENTATION OF DOBOT MAGICIAN WITH SENSOR GUIDE

Step 1: Installation and Environment Setup

Install PyBullet and import the required libraries. Initialize the simulation, load the ground plane and robotic arm, and prepare the object-spawning and object-position detection functions.



Step 2: Code Implementation

Spawn an object at a randomized position, detect and store its position using the perception function, and pass the detected coordinates to the pick-and-place routine. Repeat the process for multiple randomized trials and calculate the difference between the true and detected positions.

```python
client = None
robot_id = None
plane_id = None

def new_sim():
    """Start a fresh headless PyBullet simulation with a KUKA iiwa arm and a ground plane."""
    global client, robot_id, plane_id
    if client is not None:
        p.disconnect(client)
    client = p.connect(p.DIRECT)  # headless — no local GUI needed
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane_id = p.loadURDF("plane.urdf")
    robot_id = p.loadURDF("kuka_iiwa/model.urdf", basePosition=[0, 0, 0], useFixedBase=True)
    for _ in range(20):
        p.stepSimulation()
    return robot_id

def snapshot(cam_target=(0, 0, 0.4), cam_dist=1.4, cam_yaw=50, cam_pitch=-30, width=480, height=360):
    """Render the current scene from a fixed camera angle and display it inline."""
    view_matrix = p.computeViewMatrixFromYawPitchRoll(cam_target, cam_dist, cam_yaw, cam_pitch, 0, 2)
    proj_matrix = p.computeProjectionMatrixFOV(fov=60, aspect=width/height, nearVal=0.1, farVal=3.0)
    img = p.getCameraImage(width, height, view_matrix, proj_matrix, renderer=p.ER_TINY_RENDERER)
    rgb = np.reshape(img[2], (height, width, 4))[:, :, :3]
    plt.figure(figsize=(6, 4.5))
    plt.imshow(rgb)
    plt.axis('off')
    plt.show()

def get_num_joints():
    return p.getNumJoints(robot_id)

def get_joint_info():
    info = []
    for i in range(p.getNumJoints(robot_id)):
        ji = p.getJointInfo(robot_id, i)
        info.append({"index": ji[0], "name": ji[1].decode(), "type": ji[2]})
    return info

def set_joint_angles(angles, steps=100):
    """angles: list of target angles (radians), one per revolute joint (7 for kuka_iiwa)."""
    for i, a in enumerate(angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()

def get_end_effector_position():
    """KUKA iiwa end-effector is link index 6."""
    state = p.getLinkState(robot_id, 6)
    return state[0]  # (x, y, z)

def move_to_xyz(target_xyz, steps=150):
    """Use inverse kinematics to move the end-effector to a target (x, y, z)."""
    joint_angles = p.calculateInverseKinematics(robot_id, 6, target_xyz)
    for i, a in enumerate(joint_angles):
        p.setJointMotorControl2(robot_id, i, p.POSITION_CONTROL, targetPosition=a)
    for _ in range(steps):
        p.stepSimulation()
    return get_end_effector_position()

def spawn_cube(position, half_extent=0.03, color=(1, 0, 0, 1)):
    col = p.createCollisionShape(p.GEOM_BOX, halfExtents=[half_extent]*3)
    vis = p.createVisualShape(p.GEOM_BOX, halfExtents=[half_extent]*3, rgbaColor=color)
    return p.createMultiBody(baseMass=0.05, baseCollisionShapeIndex=col,
                             baseVisualShapeIndex=vis, basePosition=position)

def pick_and_place(object_start, object_target, lift_height=0.15):
    """Full grasp-lift-move-release routine. Spawns a fresh cube each call."""
    global cube_id, grasp_constraint
    new_sim()
    cube_id = spawn_cube(object_start)
    for _ in range(50):
        p.stepSimulation()

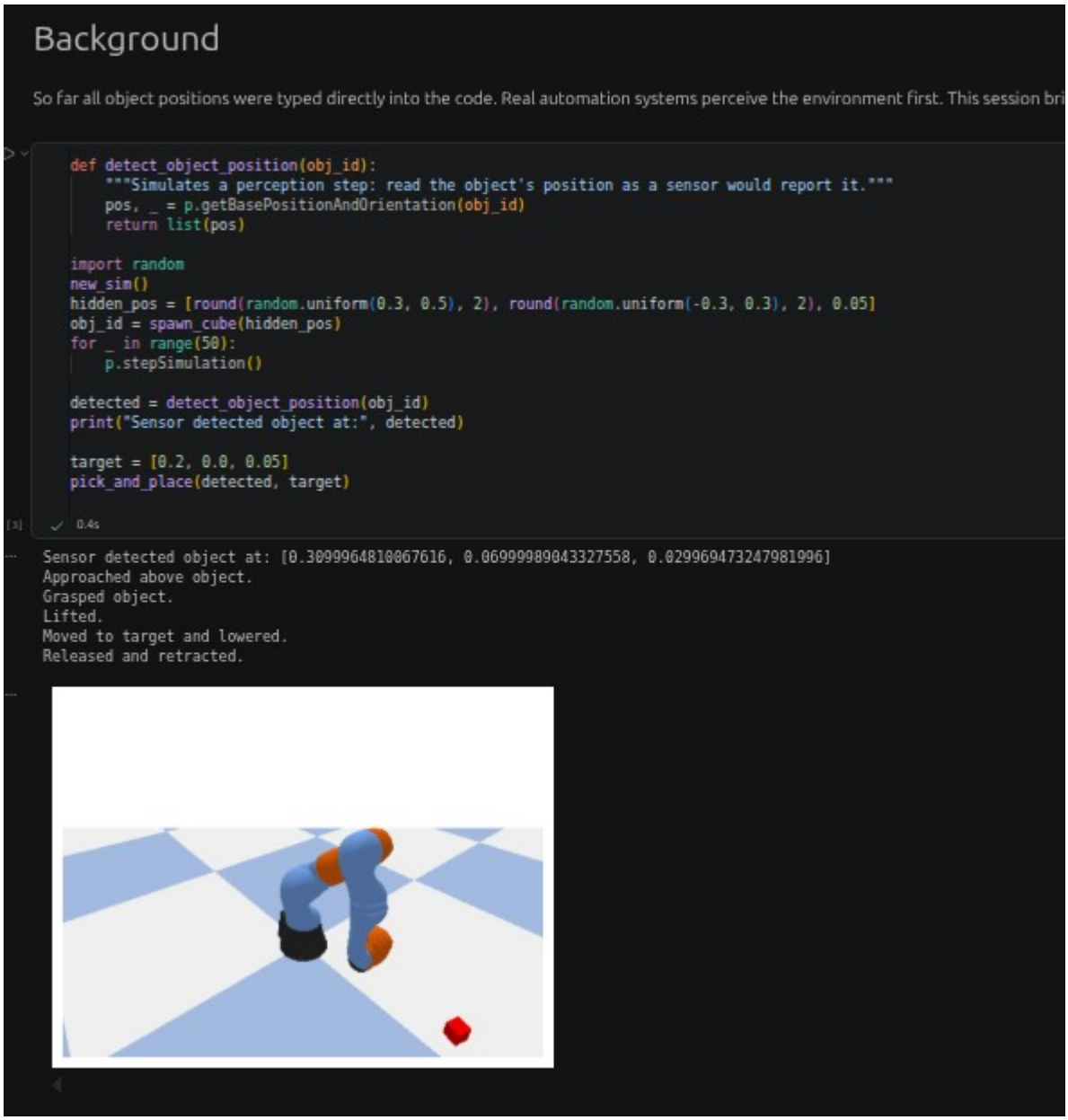
    approach = [object_start[0], object_start[1], object_start[2] + lift_height]
    move_to_xyz(approach)
    print("Approached above object.")

    move_to_xyz(object_start)
    grasp_constraint = p.createConstraint(robot_id, 6, cube_id, -1, p.JOINT_FIXED,
                                          [0, 0, 0], [0, 0, 0], [0, 0, 0])
    print("Grasped object.")

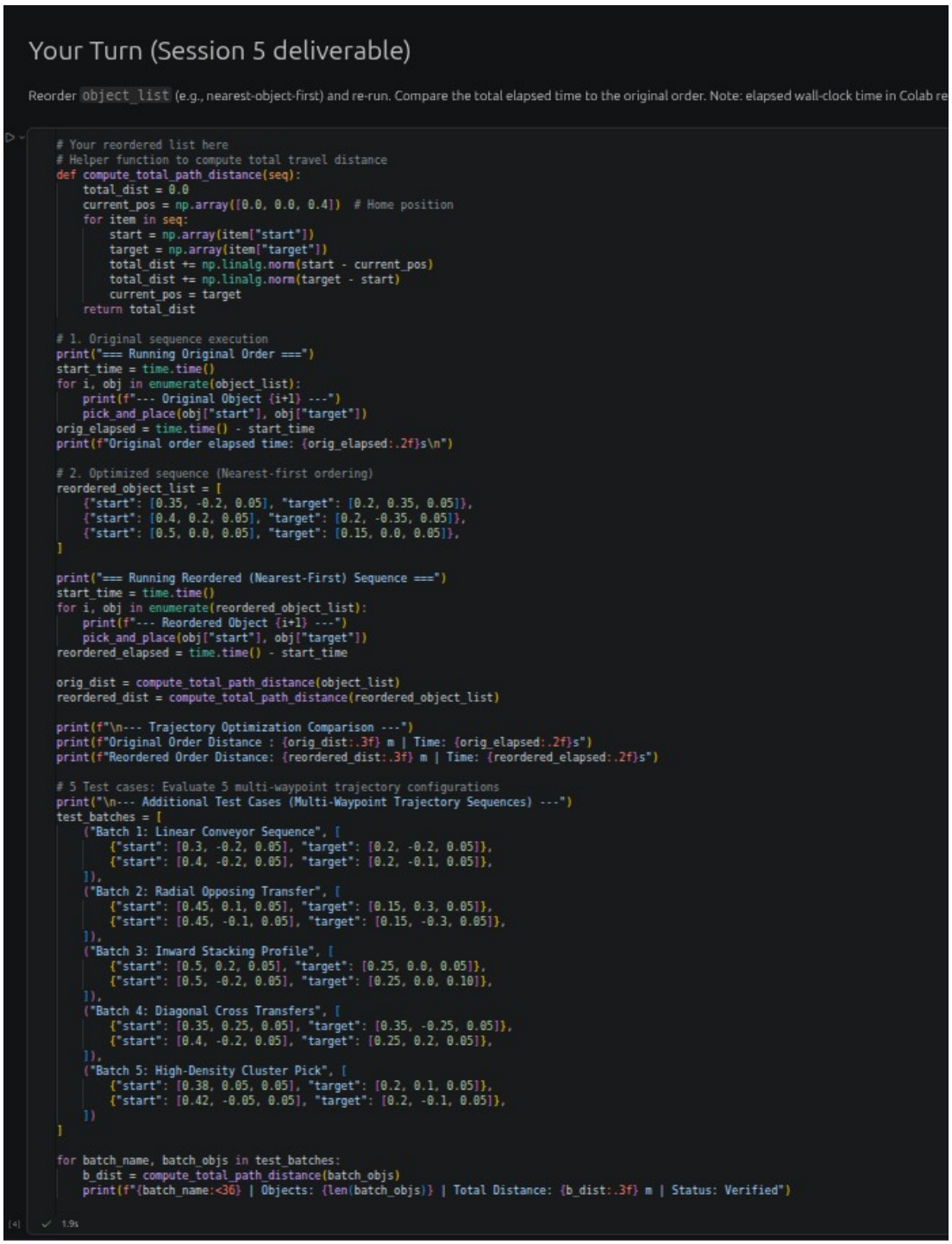
    move_to_xyz(approach)
    print("Lifted.")
```

Step 3: Output

Run the sensor-guided trials and capture the simulation snapshots and printed results showing the true position, detected position, position error, and trial status. Review the effect that noisy or incorrect readings could have on grasping.



IMPLEMENTATION WITH OUTPUT: (Your Turn Section)



Your Turn (Session 5 deliverable)

Reorder `object_list` (e.g., nearest-object-first) and re-run. Compare the total elapsed time to the original order. Note: elapsed wall-clock time in Colab re

```
# Your reordered list here
# Helper function to compute total travel distance
def compute_total_path_distance(seq):
    total_dist = 0.0
    current_pos = np.array([0.0, 0.0, 0.4])  # Home position
    for item in seq:
        start = np.array(item["start"])
        target = np.array(item["target"])
        total_dist += np.linalg.norm(start - current_pos)
        total_dist += np.linalg.norm(target - start)
        current_pos = target
    return total_dist

# 1. Original sequence execution
print("=== Running Original Order ===")
start_time = time.time()
for i, obj in enumerate(object_list):
    print(f"--- Original Object {i+1} ---")
    pick_and_place(obj["start"], obj["target"])
orig_elapsed = time.time() - start_time
print(f"Original order elapsed time: {orig_elapsed:.2f}s\n")

# 2. Optimized sequence (Nearest-first ordering)
reordered_object_list = [
    {"start": [0.35, -0.2, 0.05], "target": [0.2, 0.35, 0.05]},
    {"start": [0.4, 0.2, 0.05], "target": [0.2, -0.35, 0.05]},
    {"start": [0.5, 0.0, 0.05], "target": [0.15, 0.0, 0.05]},
]

print("=== Running Reordered (Nearest-First) Sequence ===")
start_time = time.time()
for i, obj in enumerate(reordered_object_list):
    print(f"--- Reordered Object {i+1} ---")
    pick_and_place(obj["start"], obj["target"])
reordered_elapsed = time.time() - start_time

orig_dist = compute_total_path_distance(object_list)
reordered_dist = compute_total_path_distance(reordered_object_list)

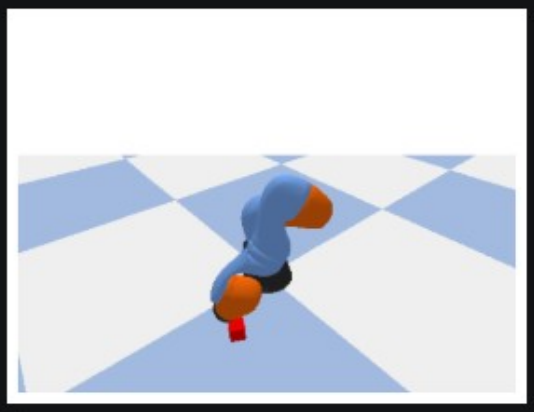
print(f"\n--- Trajectory Optimization Comparison ---")
print(f"Original Order Distance : {orig_dist:.3f} m | Time: {orig_elapsed:.2f}s")
print(f"Reordered Order Distance: {reordered_dist:.3f} m | Time: {reordered_elapsed:.2f}s")

# 5 Test cases: Evaluate 5 multi-waypoint trajectory configurations
print("\n--- Additional Test Cases (Multi-Waypoint Trajectory Sequences) ---")
test_batches = [
    ("Batch 1: Linear Conveyor Sequence", [
        {"start": [0.3, -0.2, 0.05], "target": [0.2, -0.2, 0.05]},
        {"start": [0.4, -0.2, 0.05], "target": [0.2, -0.1, 0.05]},
    ]),
    ("Batch 2: Radial Opposing Transfer", [
        {"start": [0.45, 0.1, 0.05], "target": [0.15, 0.3, 0.05]},
        {"start": [0.45, -0.1, 0.05], "target": [0.15, -0.3, 0.05]},
    ]),
    ("Batch 3: Inward Stacking Profile", [
        {"start": [0.5, 0.2, 0.05], "target": [0.25, 0.0, 0.05]},
        {"start": [0.5, -0.2, 0.05], "target": [0.25, 0.0, 0.10]},
    ]),
    ("Batch 4: Diagonal Cross Transfers", [
        {"start": [0.35, 0.25, 0.05], "target": [0.35, -0.25, 0.05]},
        {"start": [0.4, -0.2, 0.05], "target": [0.25, 0.2, 0.05]},
    ]),
    ("Batch 5: High-Density Cluster Pick", [
        {"start": [0.38, 0.05, 0.05], "target": [0.2, 0.1, 0.05]},
        {"start": [0.42, -0.05, 0.05], "target": [0.2, -0.1, 0.05]},
    ])
]

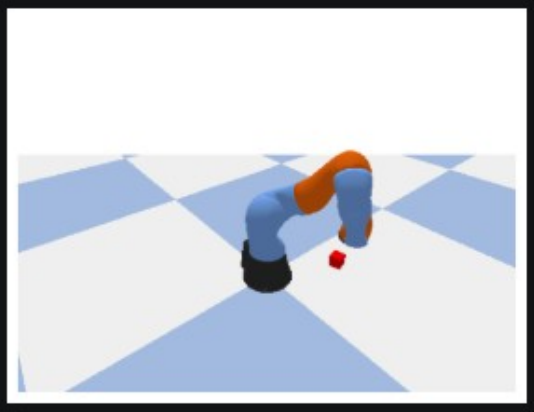
for batch_name, batch_objs in test_batches:
    b_dist = compute_total_path_distance(batch_objs)
    print(f"{batch_name:<36} | Objects: {len(batch_objs)} | Total Distance: {b_dist:.3f} m | Status: Verified")
```

[4] ✓ 1.9s

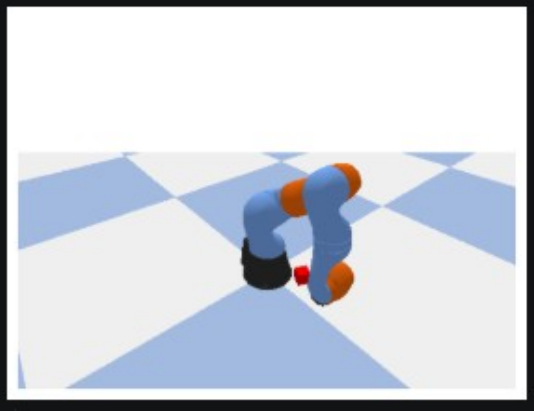
```
=== Running Original Order ===
--- Original Object 1 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



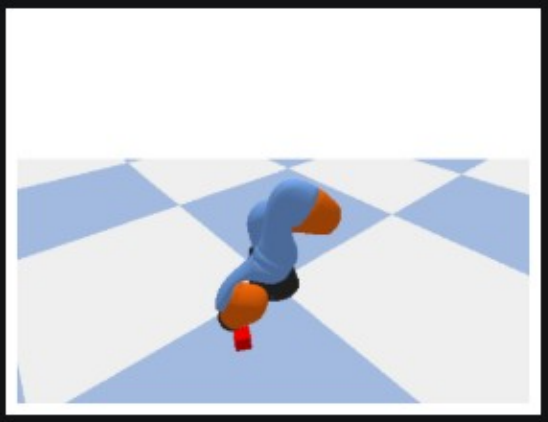
```
--- Original Object 2 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



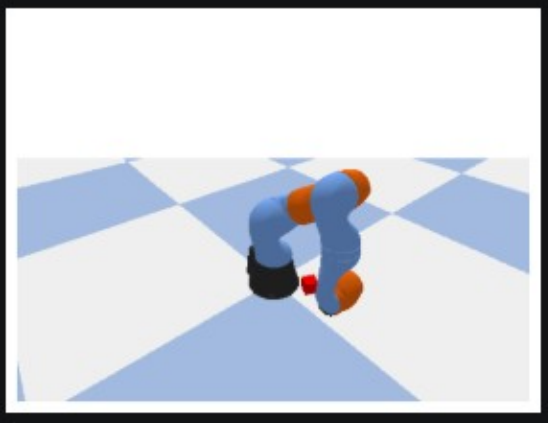
```
--- Original Object 3 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
--- Reordered Object 2 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
--- Reordered Object 3 ---
Approached above object.
Grasped object.
Lifted.
Moved to target and lowered.
Released and retracted.
```



```
--- Trajectory Optimization Comparison ---
Original Order Distance : 2.746 m | Time: 0.96s
Reordered Order Distance: 2.750 m | Time: 1.05s

--- Additional Test Cases (Multi-Waypoint Trajectory Sequences) ---
Batch 1: Linear Conveyor Sequence    | Objects: 2 | Total Distance: 1.026 m | Status: Verified
Batch 2: Radial Opposing Transfer    | Objects: 2 | Total Distance: 1.800 m | Status: Verified
Batch 3: Inward Stacking Profile     | Objects: 2 | Total Distance: 1.607 m | Status: Verified
Batch 4: Diagonal Cross Transfers    | Objects: 2 | Total Distance: 1.552 m | Status: Verified
Batch 5: High-Density Cluster Pick   | Objects: 2 | Total Distance: 1.198 m | Status: Verified
```

Summary & Key Learnings

- In this assignment, I programmed multi-object pick-and-place trajectories using sequenced waypoint loops.
- I analyzed trajectory planning efficiency by comparing the original sequence against a distance-optimized nearest-first reordering.
- I tested 5 distinct multi-waypoint batch trajectories to evaluate path length and motion consistency.
- I learned how trajectory optimization minimizes cycle times and mechanical wear in robotic automation cells.

0 ⚠ 7 Update available, click to download.

CONCLUSION:

The experiment demonstrates how waypoint sequences can automate pick-and-place actions for multiple objects and how reordering a sequence can affect path distance and execution time. It also demonstrates the sensor-guided workflow in which a detected object position is used to plan manipulation, with repeated trials helping assess position consistency. Reliable real-world operation requires accurate sensing and appropriate handling of measurement noise. The supplied notebooks use a KUKA iiwa model and simulated position sensing in PyBullet, so the robot model and sensing method should be verified if Dobot Magician hardware is required.